

Forward Kinematics of the JetCobot Arm

Based on `kinematics.py` and `jetcobot2.urdf`

Based on the provided code, the forward kinematics of the JetCobot arm computes the pose (position and orientation) of the `camera_link` frame relative to the `base_link` frame, given the six joint angles. The chain is fixed and consists of six revolute/continuous joints (`1_Joint` to `6_Joint`) followed by a fixed joint (`camera_Joint`). Below are the explicit mathematical equations.

1 Basic Homogeneous Transformation

A homogeneous transformation matrix $\mathbf{T} \in SE(3)$ is defined as:

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix},$$

where $\mathbf{R} \in SO(3)$ is a rotation matrix and $\mathbf{t} \in \mathbb{R}^3$ is a translation vector.

2 Joint Transformation

For each joint j (where $j \in \{1, \dots, 6, \text{camera}\}$), the URDF provides:

- A fixed **origin** with translation $\mathbf{t}_j = (x_j, y_j, z_j)^\top$ and Euler angles (roll, pitch, yaw) $(\phi_j, \theta_j, \psi_j)$ that define a fixed rotation $\mathbf{R}_{\text{rpy},j} = R_z(\psi_j)R_y(\theta_j)R_x(\phi_j)$.
- A rotation **axis** $\mathbf{a}_j = (a_{x,j}, a_{y,j}, a_{z,j})^\top$ (unit vector) for revolute/continuous joints.
- A joint variable θ_j (for revolute/continuous joints; for fixed joints it is ignored).

The transformation **from the parent link to the child link** through joint j is:

$$\mathbf{T}_j(\theta_j) = \underbrace{\begin{bmatrix} \mathbf{R}_{\text{rpy},j} & \mathbf{t}_j \\ \mathbf{0}^\top & 1 \end{bmatrix}}_{\text{fixed part } \mathbf{T}_{\text{fixed},j}} \cdot \underbrace{\begin{bmatrix} \mathbf{R}_{\text{axis}}(\theta_j) & \mathbf{0} \\ \mathbf{0}^\top & 1 \end{bmatrix}}_{\text{variable rotation about axis}},$$

where

$$\mathbf{R}_{\text{axis}}(\theta) = \mathbf{I} + \sin \theta [\mathbf{a}]_\times + (1 - \cos \theta) [\mathbf{a}]_\times^2$$

is the Rodrigues rotation matrix for a unit axis $\mathbf{a}$ (the axis is normalized if necessary). $[\mathbf{a}]_\times$ denotes the skew-symmetric matrix of $\mathbf{a}$.

For a **fixed joint** (like `camera_Joint`), the variable part is the identity, so

$$\mathbf{T}_{\text{camera}} = \mathbf{T}_{\text{fixed,camera}}.$$

3 Overall Kinematic Chain

The kinematic chain from `base_link` to `camera_link` is exactly:

$$\begin{aligned}
 \text{base_link} &\xrightarrow{1_Joint} 1_Link \\
 &\xrightarrow{2_Joint} 2_Link \\
 &\xrightarrow{3_Joint} 3_Link \\
 &\xrightarrow{4_Joint} 4_Link \\
 &\xrightarrow{5_Joint} 5_Link \\
 &\xrightarrow{6_Joint} 6_Link \\
 &\xrightarrow{\text{camera_Joint}} \text{camera_link}.
 \end{aligned}$$

Therefore, the total transformation from `base_link` to `camera_link` is:

$$\boxed{\mathbf{T}_{\text{base}}^{\text{cam}}(\theta_1, \dots, \theta_6) = \mathbf{T}_1(\theta_1) \mathbf{T}_2(\theta_2) \mathbf{T}_3(\theta_3) \mathbf{T}_4(\theta_4) \mathbf{T}_5(\theta_5) \mathbf{T}_6(\theta_6) \mathbf{T}_{\text{camera}} }$$

where each $\mathbf{T}_j(\theta_j)$ is defined in Section 2.

4 Pose Extraction

Given the resulting homogeneous matrix

$$\mathbf{T}_{\text{base}}^{\text{cam}} = \begin{bmatrix} \mathbf{R}_{\text{cam}} & \mathbf{p}_{\text{cam}} \\ \mathbf{0}^\top & 1 \end{bmatrix},$$

- **Position** of the camera in the `base_link` frame is simply the translation vector:

$$\mathbf{p}_{\text{cam}} = (p_x, p_y, p_z)^\top.$$

- **Orientation** is represented as a quaternion (q_x, q_y, q_z, q_w) computed from $\mathbf{R}_{\text{cam}}$ via a stable algorithm (e.g., the one implemented in the code). This quaternion is in the same frame (`base_link`).

5 Transformation to World Frame

If the robot's base is itself placed in a world frame with a known pose $(\mathbf{t}_{\text{base}}, \mathbf{R}_{\text{base}})$, then the camera pose in the world frame is:

$$\mathbf{T}_{\text{world}}^{\text{cam}} = \mathbf{T}_{\text{world}}^{\text{base}} \mathbf{T}_{\text{base}}^{\text{cam}},$$

where

$$\mathbf{T}_{\text{world}}^{\text{base}} = \begin{bmatrix} \mathbf{R}_{\text{base}} & \mathbf{t}_{\text{base}} \\ \mathbf{0}^\top & 1 \end{bmatrix}.$$

The world-frame position and quaternion are then extracted from $\mathbf{T}_{\text{world}}^{\text{cam}}$.

6 Explicit Symbolic Expression for Each Joint

For a concrete implementation, each $\mathbf{T}_j$ can be written in full matrix form. For a revolute joint with axis $\mathbf{a} = (a_x, a_y, a_z)$ (unit) and fixed transform given by translation $\mathbf{t} = (t_x, t_y, t_z)$ and roll-pitch-yaw (ϕ, θ, ψ) :

$$\mathbf{T}_j(\theta) = \begin{bmatrix} R_z(\psi)R_y(\theta)R_x(\phi) & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \begin{bmatrix} \mathbf{R}_{\text{axis}}(\theta) & \mathbf{0} \\ \mathbf{0}^\top & 1 \end{bmatrix},$$

with

$$\mathbf{R}_{\text{axis}}(\theta) = \begin{bmatrix} c + a_x^2(1-c) & a_x a_y(1-c) - a_z s & a_x a_z(1-c) + a_y s \\ a_y a_x(1-c) + a_z s & c + a_y^2(1-c) & a_y a_z(1-c) - a_x s \\ a_z a_x(1-c) - a_y s & a_z a_y(1-c) + a_x s & c + a_z^2(1-c) \end{bmatrix},$$

where $s = \sin \theta$, $c = \cos \theta$.

The numerical values of $\mathbf{t}_j$, $(\phi_j, \theta_j, \psi_j)$, and $\mathbf{a}_j$ are read directly from the URDF file (`jetcobot2.urdf`) for each joint in the chain.

7 Summary of the Algorithm

1. For each joint j in the chain, retrieve its fixed origin (xyz, rpy) and axis.
2. Build the fixed transformation $\mathbf{T}_{\text{fixed},j}$ from the origin data.
3. Build the variable rotation $\mathbf{R}_{\text{axis}}(\theta_j)$ from the axis and the joint angle.
4. Form $\mathbf{T}_j = \mathbf{T}_{\text{fixed},j} \cdot \text{diag}(\mathbf{R}_{\text{axis}}, 1)$.
5. Multiply all $\mathbf{T}_j$ in order (including the fixed camera joint) to obtain the overall transform.
6. Extract position and quaternion from the final matrix.

This is precisely what the `JetCobotKinematics.camera_pose_in_base` method implements. The equations above formalize the computation mathematically.